

1. what is public static void main explain in detail

In Java programming, `public static void main(String[] args)` is the entry point of any standard Java application. When you run a Java program, the Java Virtual Machine (JVM) looks specifically for this exact method signature to start executing the code.

Here is a detailed breakdown of each component that makes up this famous line of code.

The Signature Breakdown

Java

```
public static void main(String[] args)
```

Each keyword and component plays a specific role in how the JVM interacts with your program:

1. **public (Access Modifier)**

What it means: `public` is an access modifier that makes the method visible and accessible from anywhere, even outside its class package.

Why it's needed: The JVM resides outside of your class (it is part of the Java runtime environment). To execute the `main` method from outside the class, the JVM needs unrestricted access to it. If it were `private` or `protected`, the JVM wouldn't be able to call it.

2. **static (Keyword)**

What it means: The `static` keyword means that the method belongs to the class itself, rather than to any specific instance (object) of the class.

Why it's needed: When your Java application starts, no objects of your class have been created yet. If the `main` method weren't `static`, the JVM would have to instantiate (create an object of) your class first before it could call `main`. By making it `static`, the JVM can invoke the method directly using just the class name, saving memory and avoiding the need for initialization.

3. void (Return Type)

What it means: void specifies that the method does not return any value back to the caller.

Why it's needed: Once the main method finishes executing, the Java program terminates. There is nowhere for a return value to go (since the JVM is just shutting down the process), so it returns void.

4. main (Method Name)

What it means: This is the exact name of the method that the JVM looks for as the starting point of execution.

Why it's needed: It is a hardcoded identifier built into the JVM specification. If you name it anything else (like start or run), the JVM will throw a `NoSuchMethodError` because it won't know where to begin running your program.

5. String[] args (Parameters / Arguments)

What it means: This accepts an array of String objects, which allows the program to receive command-line arguments passed when running the program from a terminal or command prompt.

Why it's needed:

`String[]` is an array of strings.

`args` is simply the variable name (short for "arguments"), though you could technically name it anything you like (e.g., `String[] parameters`).

If you run a program like `java MyProgram hello world`, the strings "hello" and "world" are stored in the `args` array and can be used inside your program.

Summary of Execution Flow

You compile your Java file (`javac MyClass.java`), creating bytecode (`MyClass.class`).

You run the program using the JVM (`java MyClass`).

The JVM loads `MyClass` into memory.

The JVM searches for `public static void main(String[] args)`.

2.DIFFERENCES BETWEEN PRIMITIVE AND NONPRIMITIVE

The core differences between primitive and non-primitive (reference) data types come down to how they store data, where they live in memory, and how they behave:

Storage & Memory:

- Primitive: Stores the actual value directly in the Stack memory.
- Non-Primitive: Stores a memory address (reference) pointing to the actual object located in the Heap memory.

Nullability:

- Primitive: Cannot be null. They always contain a default value (like 0 or false).
- Non-Primitive: Can be null if they are not initialized to point to an object.

Methods and Behavior:

- Primitive: Have no built-in methods; they are just raw data (e.g., you cannot call `.length()` on a primitive `int`).
- Non-Primitive: Can call methods because they are objects (e.g., `myString.toUpperCase()`).

Naming and Syntax:

- Primitive: Built-in keywords written in lowercase (e.g., `int`, `char`, `boolean`).
- Non-Primitive: Usually start with an uppercase letter and are created by programmers or the language framework (e.g., `String`, `Integer`, `Arrays`).

Size:

- Primitive: Have a fixed size determined strictly by their type (e.g., a `long` is always 8 bytes).
- Non-Primitive: Size can vary depending on the complexity of the data inside the object.

3.Classes and Constructors in Object-Oriented Programming

1. What is a Class?

A class is a user-defined blueprint or prototype from which objects are created. It serves as a template that defines the attributes (variables/fields) and behaviors (methods/functions) that the objects of its type will have.

Attributes: The data or state (e.g., `name`, `age`, `color`).

Methods: The actions or functions the object can perform (e.g., `drive()`, `speak()`).

2. What is a Constructor?

A constructor is a special member method (or function) of a class that is automatically executed when an object of that class is created.

Primary Purpose: To initialize the object's instance variables (allocate values to the object's attributes).

Key Characteristics:

- It shares the exact same name as the class.
- It does not have an explicit return type (not even void).
- It is called automatically when using the new keyword (in languages like Java, C++, C#, etc.).

3. Ways of Transferring Data Through Objects

In object-oriented programming, data is passed into, out of, or between objects using several standard approaches. Depending on the language (like Java, C++, or Python), there are four primary ways to transfer data through an object:

Way 1: Through Constructors (Constructor Injection)

Data is passed directly to the object at the exact moment of its creation.

How it works: Arguments are passed into the constructor parameters, which then assign those values to the object's instance variables.

Use Case: Best for mandatory data required for the object to exist in a valid state.

Example (Java-style pseudo-code):

Way 2: Through Setter Methods (Property/Mutator Injection)

Data is transferred to an existing object after it has already been created, typically by calling dedicated methods (setters).

How it works: The object is instantiated with default or empty values, and data is injected later using methods designed to modify state.

Use Case: Ideal for optional data or when data needs to be updated dynamically during the lifetime of the object.

Way 3: Through Method Arguments (Behavioral Transfer)

Data is transferred dynamically by passing an object as an argument into another method (either belonging to the same object or a different utility class/service).

How it works: The object packages its data or is itself passed into a function to process or exchange data with other parts of the program.

Use Case: Sharing state between different objects or performing operations involving multiple objects.

4.What is the Object Class?

In object-oriented programming languages like Java and C#, the Object class is the ultimate root or the mother of all classes.

Every single class you create—whether built-in or user-defined—automatically inherits from the Object class, either directly or indirectly. If a class does not explicitly extend another class, the compiler implicitly makes it a child of the Object class.

5. Top 4 Methods of the Object Class

- `toString()`: Returns a string representation of the object (overridden to show readable data).
- `equals(Object obj)`: Compares two objects (by default checks memory references; often overridden to compare content).
- `hashCode()`: Returns an integer hash code used by collections like `HashMap` and `HashSet`.
- `getClass()`: Returns the exact runtime class type of the object for reflection.

6. What is Inheritance in Java?

Inheritance is a fundamental concept in Object-Oriented Programming (OOP) that allows one class (called the child class or subclass) to inherit the fields (variables) and methods of another class (called the parent class or superclass).

The Core Benefit: It promotes code reusability. Instead of writing the same code repeatedly, you can write common features in a parent class and inherit them in multiple child classes.

The Keyword: In Java, inheritance is achieved using the `extends` keyword.

Types of Inheritance in Java

Java supports four main types of inheritance.

1. Single Inheritance

In single inheritance, a single child class extends a single parent class. This is the simplest and most common form of inheritance.

- Structure: Class B inherits from Class A.

- **Example:**

Java

```
class Animal { void eat() { System.out.println("Eating..."); } }
```

```
class Dog extends Animal { void bark() { System.out.println("Barking..."); } }
```

2. Multilevel Inheritance

In multilevel inheritance, a class is derived from a class which is also derived from another class (a chain of inheritance).

- Structure: Class C inherits from Class B, which inherits from Class A (A ---->B ---->C).

- **Example:**

Java

```
class Animal { void eat() { /* ... */ } }
```

```
class Dog extends Animal { void bark() { /* ... */ } }
```

```
class BabyDog extends Dog { void weep() { System.out.println("Weeping..."); } }
```

3. Hierarchical Inheritance

In hierarchical inheritance, multiple child classes inherit from a single parent class.

- Structure: Class B, Class C, and Class D all inherit from the same parent Class A.

Example:

Java

```
class Animal { void eat() { /* ... */ } }
```



```
class Dog extends Animal { void bark() { /* ... */ } }
```

```
class Cat extends Animal { void meow() { System.out.println("Meowing..."); } }
```

4. Multiple Inheritance (via Interfaces)

As mentioned earlier, Java does not allow a class to extend multiple classes directly. But a single class can implement multiple interfaces, allowing a form of multiple inheritance.

- Structure: Class C implements Interface A and Interface B.
- **Example:**

Java

```
interface Printable { void print(); }
```

```
interface Showable { void show(); }
```

```
class Document implements Printable, Showable {  
    public void print() { System.out.println("Printing..."); }  
    public void show() { System.out.println("Showing..."); }  
}
```

7. What is Encapsulation?

Encapsulation is one of the core principles of Object-Oriented Programming (OOP). It is the practice of wrapping data (variables) and code acting on the data (methods) together as a single unit, while restricting direct access to some of the object's components.

- How it works: You make class variables private (hidden from the outside world) and provide public getter and setter methods to safely view or modify that data.
- Benefits: Data security, controlled access, and flexibility.

- **Example: Bank and Person**

Imagine a scenario where a Person wants to interact with a Bank account. Encapsulation ensures that the person cannot directly change their account balance; they must go through controlled bank operations (methods).